
**ClimbIt: Herramienta para gestionar y gamificar
el progreso en rocódromos**
**ClimbIt: Tool to manage and gamify progress on
climbing walls**



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Iván Alcalde Cámara
Ignacio Melendo Bruggeman

Director

Iván Martínez Ortiz

Grado en Ingeniería del Software
Facultad de Informática
Universidad Complutense de Madrid

ClimbIt: Herramienta para gestionar y
gamificar el progreso en rocódromos
ClimbIt: Tool to manage and gamify
progress on climbing walls

Trabajo de Fin de Grado en Ingeniería del Software

Autor

**Iván Alcalde Cámara
Ignacio Melendo Bruggeman**

Director

Iván Martínez Ortiz

Convocatoria: Junio de 2026

**Grado en Ingeniería del Software
Facultad de Informática
Universidad Complutense de Madrid**

25 de Mayo de 2026

Dedicatoria

*A la maravillosa cafetería de la Facultad de
Informática, por ser el lugar donde se gestó
esta memoria y por su inestimable
contribución a lo largo de todo el proceso.*

Agradecimientos

A todos los que nos han acompañado por nuestro lento pero bonito paso por la universidad.

Resumen

ClimbIt: Herramienta para gestionar y gamificar el progreso en rocódromos

Un resumen en castellano de media página, incluyendo el título en castellano. A continuación, se escribirá una lista de no más de 10 palabras clave.

Palabras clave

Máximo 10 palabras clave separadas por comas

Abstract

ClimbIt: Tool to manage and gamify progress on climbing walls

An abstract in English, half a page long, including the title in English. Below, a list with no more than 10 keywords.

Keywords

10 keywords max., separated by commas.

Índice

1. Introducción	1
1.1. Contexto de la escalada en rocódromo	1
1.1.1. Funcionamiento de la escalada en rocódromo	1
1.1.2. Tipos de rutas en rocódromos	1
1.1.3. Dificultad y progreso	2
1.2. Contexto de la gamificación en deportes	2
1.2.1. Modelo psicológico RAMP	2
1.2.2. El sistema PBL (<i>Points, Badges, Leaderboards</i>)	3
1.2.3. Beneficios de la gamificación en el deporte	3
1.3. Motivación	4
1.4. Objetivos	4
1.4.1. Objetivo general	4
1.4.2. Hitos específicos	4
1.5. Estructura del documento	5
2. Estado del Arte	7
2.1. Análisis de aplicaciones de escalada	7
2.1.1. Aplicaciones integradas con el rocódromo	7
2.1.2. Aplicaciones centradas en la comunidad	8
2.1.3. Aplicaciones con hardware específico	8
2.2. Análisis de aplicaciones de otros deportes	9
2.2.1. Competiciones y ranking por segmentos (Strava)	9
2.2.2. Coleccionismo de logros (Pokémon Go)	9
2.3. Conclusiones del Estado del Arte e identificación de la oportunidad	10
2.3.1. Síntesis de la problemática actual	10
2.3.2. Identificación de la oportunidad	10
3. Metodología de Desarrollo y Planificación	13
3.1. Metodología de trabajo: Scrumban	13
3.2. Proceso de Captura de Requisitos y Definición del Producto	13
3.2.1. Lienzo de Propuesta de Valor (Value Proposition Canvas)	14
3.2.2. Visualización del viaje: User Story Map	14

3.3.	Herramientas de gestión y seguimiento	15
3.4.	Gestión de la Configuración y Control de Versiones	15
3.5.	Uso de Inteligencia Artificial como Apoyo al Desarrollo	16
4.	Análisis y Especificación de Requisitos	17
4.1.	Personas	17
4.1.1.	Persona 1: Klaudia, Escaladora Constante	17
4.2.	Roles	17
4.2.1.	Escalador	17
4.2.2.	Gestor de Rocódromo	17
4.3.	Especificación de funcionalidades (Historias de Usuario / Casos de Uso)	17
4.4.	Requisitos no funcionales	18
4.5.	Alcance de las versiones	18
4.5.1.	Requisitos del prototipo (v.0.1.0)	18
4.5.2.	Requisito del MVP (v.1.0.0)	18
5.	Diseño del Sistema y Decisiones Técnicas	19
5.1.	Decisiones de Diseño y Stack Tecnológico	19
5.2.	Diseño de la Arquitectura	20
5.2.1.	Backend	20
5.2.2.	Frontend	22
5.3.	Diseño de la Base de Datos	22
5.4.	Diseño de la UI y UX	22
6.	Implementación y Desarrollo	23
6.1.	Desarrollo del Backend	23
6.2.	Desarrollo del Frontend	23
6.3.	Desafíos técnicos encontrados y soluciones	23
7.	Integración Continua, Despliegue y DevOps (CI/CD)	25
7.1.	Diseño del Pipeline de CI/CD	25
7.2.	Entornos de desarrollo, preproducción y producción	25
7.3.	Despliegue de la infraestructura	25
7.4.	Empaquetado y Subida a Google Play	25
8.	Pruebas y Validación	27
8.1.	Estrategia de pruebas técnicas	27
8.2.	Pruebas con usuarios reales (User Testing)	27
8.3.	Análisis de Resultados y Mejoras aplicadas	27
9.	Conclusiones y Trabajo Futuro	29
	Introduction	31
	Conclusions and Future Work	33

Contribuciones Personales	35
Bibliografía	37
A. Título del Apéndice A	39
B. Título del Apéndice B	41

Índice de figuras

Índice de tablas

3.1. Anatomía de configuración de una <i>Issue</i> en GitHub Projects.	15
--	----

Capítulo 1

Introducción

En este capítulo se introducen los conceptos básicos del proyecto, centrándonos en la escalada en rocódromo y en el uso de técnicas de gamificación aplicadas al deporte. Posteriormente se justifican los la motivación y objetivos de este preyecto y se resume la estructura de la memoria.

1.1. Contexto de la escalada en rocódromo

La escalada es un deporte con numerosas modalidades de las cuales nos centraremos en la "indoor" que se practica en rocódromos. A continuación se detalla su funcionamiento, tipos de ruta y el sistema de dificultad.

1.1.1. Funcionamiento de la escalada en rocódromo

Un rocódromo es una instalación deportiva que emula la escalada en roca natural mediante muros artificiales equipados con presas (agarres) de diferentes formas, tamaños y colores. El objetivo del escalador es ascender por una ruta predefinida usando exclusivamente las presas designadas a esta ruta.

Las rutas se identifican mediante un mismo color de presas. Cada ruta tiene un punto de inicio y un punto final o *top* (última presa que se debe agarrar con ambas manos para completar el ascenso). Cuando una ruta se completa en el primer intento, se denomina *flash*.

A diferencia de la escalada en roca natural, las rutas de los rocódromos son dinámicas. Con el objetivo de dar variedad y nuevos desafíos a los escaladores las rutas se van renovando periódicamente. Esta labor es llevada a cabo por los *route setter* (o equipadores), profesionales encargados de diseñar y montar los nuevos bloques y vías.

1.1.2. Tipos de rutas en rocódromos

Las principales modalidades practicadas en rocódromo son la escalada de bloque (*boulder*) y la escalada deportiva (vía):

1. **Escalada de bloque (búlder).** Se practica en muros de baja altura, normalmente de hasta 4 o 5 metros con colchonetas para la seguridad. El objetivo de este tipo de rutas es buscar una solución para problemas con secuencias de movimientos cortos e intensos.
2. **Escalada deportiva (vía).** Se realiza en muros de mayor altura por lo que se necesita material de seguridad como cuerda, arnés y sistema de aseguramiento. En estos tipos de ruta se requiere completar recorridos largos donde predominan la resistencia y la gestión del esfuerzo. Durante el ascenso, el escalador engancha la cuerda en anclajes intermedios y el asegurador, desde el suelo, controla la cuerda para detener posibles caídas.

1.1.3. Dificultad y progreso

Cada ruta, tanto de bloque como de vía, suele tener asignado un grado de dificultad que permite estimar el nivel exigido para completarlo. Este grado suele aparecer en la etiqueta de inicio.

En bloque se utilizan, entre otras, la escala de Fontainebleau (por ejemplo, 6A+, 7B) y la *V-Scale* (por ejemplo, V4, V6). En cambio en vía son frecuentes la escala francesa similar a Fontainebleau en notación y la YDS (por ejemplo, 5.10a, 5.11c). Una explicación detallada de estas escalas y sus conversiones puede consultarse en¹. Existen también análisis estadísticos de ascensos que aportan referencias de progresión, aunque no siempre son totalmente representativos².

Además de estas escalas, muchos rocódromos emplean sistemas internos basados en colores para asociar rangos de dificultad a cada ruta. Estos sistemas facilitan la lectura del nivel y pueden servir como elemento de motivación al permitir visualizar avances.

1.2. Contexto de la gamificación en deportes

La gamificación consiste en aplicar principios y componentes propios de los juegos en contextos no lúdicos, entrenamiento deportivo. El objetivo es incrementar la motivación, mejorar la retención de usuarios y favorecer la adherencia al deporte.

1.2.1. Modelo psicológico RAMP

Una de las bases principales para diseñar experiencias gamificadas es el modelo psicológico RAMP (*Relatedness, Autonomy, Mastery, Purpose*), alineado con teorías de motivación intrínseca. En este modelo se destacan cuatro necesidades principales de los usuarios:

- **Relatedness (vinculación):** necesidad del usuario de tener conexiones sociales, reconocimiento y pertenencia a una comunidad.

¹<https://www.lacrux.com/es/escalar/Escalas-de-escalada-explicadas-UIAA-Fontainebleau-V-Grado-CC>

²<https://www.alessandromasullo.com/blog/analysis-of-4-million-climbing-ascents/>

- **Autonomy (autonomía):** necesidad del usuario de sentir que el tiene el control y toma decisiones para progresar.
- **Mastery (maestría):** necesidad de sentir una mejora continua mediante objetivos alcanzables y progresivos.
- **Purpose (finalidad):** necesidad de sentir que el esfuerzo tenga un proposito más alla de la tarea inmediata.

Más información sobre motivación y gamificación puede consultarse en³.

1.2.2. El sistema PBL (*Points, Badges, Leaderboards*)

Basado en el modelo RAMP surge el sistema PBL. Este es una implementación clásica de gamificación y se compone de tres elementos principales:

- **Points (puntos):** representan el esfuerzo o rendimiento del usuario (por ejemplo, dificultad encadenada, número de intentos o constancia).
- **Badges (insignias):** representan logros alcanzados y visualizan la evolución del deportista (por ejemplo, completar 100 rutas o lograr tu primera ruta de dificultad 7a).
- **Leaderboards (clasificaciones):** comparan resultados entre usuarios, incentivando una competitividad sana y el sentido de comunidad.

Una explicación más detallada del modelo PBL se describe en⁴.

1.2.3. Beneficios de la gamificación en el deporte

La integración de mecánicas de gamificación en aplicaciones deportivas aporta beneficios relevantes:

- **Mayor retención:** la retroalimentación constante reduce el abandono y mejora la constancia del entrenamiento.
- **Visualización del progreso:** el registro de datos permite visualizar la evolución mediante métricas y logros.
- **Refuerzo de comunidad:** la interacción social aumentan la implicación de los usuarios con el deporte y su comunidad.

En un deporte individual como la escalada, donde el progreso puede ser lento y hay mucho estancamiento, estas mecánicas resultan especialmente útiles para mantener la motivación y constancia en el deporte.

³<https://playmotiv.com/gamificacion-y-motivacion/>

⁴https://www.adrformacion.com/knowledge/educacion-y-formacion/la_gamificacion_y_la_triada_pbl.html

1.3. Motivación

La escalada es un deporte unipersonal en el que el seguimiento del progreso suele realizarse de forma manual (libretas, memoria o aplicaciones externas no integradas en el propio rocódromo). La motivación principal de este proyecto es digitalizar el deporte en ciertos centros mediante una aplicación centralizada.

La aplicación permite registrar rutas completadas específicas de cada rocódromo, conservando el registro de actividad para analizar la evolución del usuario. Además, incorpora elementos de gamificación como estadísticas o Leaderboards para transformar el entrenamiento en una experiencia más social y motivadora, favoreciendo la constancia y una competitividad saludable entre la comunidad de escalada.

1.4. Objetivos

En esta sección se definen los objetivos que guían el desarrollo del proyecto, haciendo una distinción entre la meta global y los hitos técnicos necesarios.

1.4.1. Objetivo general

Desarrollar una aplicación compatible con dispositivos móviles que permita a los usuarios de un rocódromo registrar su actividad de escalada (bloque y vía) y que, mediante técnicas de gamificación, fomente la motivación y la constancia del entrenamiento.

1.4.2. Hitos específicos

1. Implementar un sistema de gestión de usuarios que cubra registro, autenticación y perfil personal con información de sus datos y estadísticas.
2. Desarrollar un sistema para escaladores de registro del estado de rutas que permita cambiar entre *flash*, completadas o proyecto.
3. Desarrollar un sistema de gestión de rutas para los *route setters* de manera que se reduzca al mínimo el esfuerzo necesario para registrar los cambios en el rocódromo.
4. Incorporar mecánicas de gamificación orientadas a mejorar la motivación del usuario.
5. Implementar funcionalidades sociales (amistades, comparación de progreso y estadísticas) para fomentar la competitividad sana en la comunidad.
6. Diseñar una interfaz y una experiencia de usuario simples, intuitivas y eficientes, preparadas para usuarios con distintos niveles tecnológicos.

1.5. Estructura del documento

Esta memoria se organiza en nueve capítulos que detallan el proceso completo de planificación, diseño y desarrollo de la aplicación. A continuación se resume brevemente el contenido de cada uno:

- **Capítulo 1: Introducción.** Presenta el contexto de la escalada *indoor*, los fundamentos de la gamificación (modelo RAMP y sistema PBL) y la motivación y los objetivos del trabajo.
- **Capítulo 2: Estado del Arte.** Realiza un análisis comparativo de aplicaciones de escalada y de otros deportes, identificando la oportunidad de mercado en la que se asienta esta propuesta.
- **Capítulo 3: Metodología de Desarrollo y Planificación.** Detalla el marco de trabajo Scrumban, las herramientas utilizadas y el rol de la inteligencia artificial como herramienta al desarrollo.
- **Capítulo 4: Análisis y Especificación de Requisitos.** Se definen los roles de usuario y las funcionalidades clave mediante historias de usuario.
- **Capítulo 5: Diseño del Sistema y Decisiones Técnicas.** Expone el *stack* tecnológico, la arquitectura del sistema (*backend* y *frontend*), el diseño de la base de datos y la interfaz de usuario.
- **Capítulo 6: Implementación y Desarrollo.** Se describe el proceso de desarrollo de los componentes software y los retos técnicos superados.
- **Capítulo 7: Integración Continua, Despliegue y DevOps.** Explica la configuración del *pipeline* de CI/CD, la gestión de infraestructura y el proceso de publicación en Google Play.
- **Capítulo 8: Pruebas y Validación.** Recoge la estrategia de las pruebas realizadas y los resultados de las sesiones con usuarios reales.
- **Capítulo 9: Conclusiones y Trabajo Futuro.** Se determinan los resultados obtenidos, se evalúa el cumplimiento de los objetivos definidos además de proponer una serie de tareas para el futuro del proyecto.

El documento finaliza con las contribuciones personales del autor, la bibliografía consultada y los apéndices correspondientes.

(por definir)

Capítulo 2

Estado del Arte

Una vez determinado el objetivo general de este proyecto, continuaremos en este capítulo analizando el trasfondo y estado del arte en dos áreas clave: aplicaciones específicas de escalada y mecánicas de gamificación aplicadas en otros deportes.

Se presenta primero un análisis del panorama tecnológico actual examinando aplicaciones relevantes, sus funcionalidades y debilidades. Posteriormente se definen los principios de diseño y mecánicas de gamificación que se pueden extrapolar para el desarrollo de la aplicación.

2.1. Análisis de aplicaciones de escalada

En esta sección se analizan las principales aplicaciones que se emplean para registrar la actividad en la escalada. Las aplicaciones se han agrupado en base a su propósito definido.

2.1.1. Aplicaciones integradas con el rocódromo

Estas aplicaciones tienen una integración estrecha con los rocódromos, actuando como una herramienta para el registro de rutas y sesiones.

TopLogger (Top, 2026): TopLogger es una aplicación móvil que permite registrar el estado con las rutas de los rocódromos asociados y facilita la identificación de estas por ubicación, color y grado. Para determinar las posiciones de las rutas en un rocódromo realiza a través de un mapa cenital que permite moverte y seleccionar las rutas deseadas rápidamente. En el perfil se pueden visualizar numerosas estadísticas sobre datos de las sesiones de escalada y para su capa de gamificación tienen implementado un sistema de niveles y clasificaciones por rocódromo.

Entre sus limitaciones la primera que salta a la vista es la sobrecarga de información en cada pantalla, haciendo poco intuitiva y poco accesible la aplicación para perfiles con un nivel tecnológico bajo. En cuanto a la gamificación, al hacer las clasificaciones por rocódromo se puede engañar a la aplicación y posicionarte primero, consiguiendo el efecto adverso de motivación para los escaladores legítimos. Además

de esto, cuenta con poca integración con los rocódromos en España y con una capa de pago si tu rocodromo cuenta con muchos escaladores habituales.

WeClimb (WeC, 2026): WeClimb presenta funcionalidades similares a TopLogger, incluyendo un registro de rutas por color y grado pero difiriendo en la posición de las rutas. En esta aplicación para seleccionar la ruta usa un sistema de tarjetas de manera que seleccionando una sección de un mapa, se muestra una serie de tarjetas con las fotos de cada ruta facilitando el reconocimiento visual. En las imágenes de las rutas existe un mecanismo para poder resaltar las presas(agarres) que se pueden usar, dando la posibilidad de crear retos personalizados. Además de esto la aplicación cuenta con una pestaña para eventos, un historial con las tarjetas de las rutas completadas y clasificaciones por rocódromo.

Como limitaciones encontramos que su despliegue está centrado para Francia por lo que no hay centros ni traducción de la app. Actualmente carece de una capa de gamificación profunda, a excepción de las clasificaciones, pero que como en TopLogger se pueden engañar creando un efecto adverso. Por último el sistema de tarjetas para las rutas puede ser algo tedioso si hay numerosas rutas en una zona haciendo lento el proceso de selección de estas.

2.1.2. Aplicaciones centradas en la comunidad

En este grupo se prioriza la conexión entre escaladores frente al registro de todos los ascensos. Funcionan como redes sociales especializadas para compartir los ascensos más importantes.

KAYA(KAY, 2026) KAYA se posiciona como una red social para escaladores (especialmente orientada a escalada en roca natural), centrada en compartir ascensos con formato de publicaciones, interactuar con usuarios y seguir la progresión de otros. Las publicaciones pueden ser registros de una ruta, registro de una sesión de escalada o subir un video corto en formato vertical de una solución. En cuanto a elementos gamificación tiene un sistema de Badges en base al número de rutas registradas en la aplicación motivando a los escaladores a registrar más rutas.

Como limitaciones determinamos que la aplicación está más orientada al registro de rutas relevantes para el escalador más que el registro completo de toda su actividad. Además, la aplicación esta principalmente centrada en escalada en roca natural por lo que tiene poca integración con rocódromos.

2.1.3. Aplicaciones con hardware específico

Existen soluciones que combinan software con elementos físicos del rocódromo para crear experiencias de entrenamiento más interactivas.

MoonBoard(Moo, 2026) MoonBoard es una instalación física de entrenamiento que combina un muro de escalada con una aplicación. La particularidad de este sistema reside en que la posición de las presas y la inclinación del muro son idénticas en

todas sus instalaciones, permitiendo así una experiencia común en cualquier centro de escalada. La aplicación se conecta al muro y permite elegir entre miles de rutas que los escaladores han creado y una vez seleccionado uno, las presas que se pueden usar se iluminan. MoonBoard tiene un sistema de ranking global que funciona por un sistema de ascensos validados por la comunidad permitiendo a los usuarios competir con escaladores de otros países y visualizar el progreso en comparación con la media mundial.

Como limitaciones destacamos que MoonBoard es un sistema diseñado para entrenamientos de alta intensidad por lo que no es accesible a escaladores novatos. Además de esto MoonBoard es un sistema cerrado por lo que únicamente se pueden registrar las rutas completadas en el funcionando como una herramienta de registro parcial.

2.2. Análisis de aplicaciones de otros deportes

En esta sección se examinan mecánicas de gamificación probadas en otras disciplinas y aplicaciones de fitness con el objetivo de identificar ideas transferibles al contexto del rocódromo.

2.2.1. Competiciones y ranking por segmentos (Strava)

Strava(Str, 2026) Strava es una aplicación de registro de sesiones de corredores y ciclistas. Parte de su éxito viene de su concepto de segmentos, que son tramos específicos de una ruta donde los usuarios registran y comparan los tiempos que han tardado en completarlo mediante GPS. Este sistema transforma un entrenamiento individual en una competición virtual y el modelo se puede extrapolar a la escalada tratando cada vía o bloque como un segmento, donde se pueden medir el número de intentos o tiempo. En cuanto a la gamificación tiene un sistema de Badges que son títulos como KOM/QOM (King/Queen of the Mountain) que fomentan la motivación de los usuarios.

Como limitaciones en el enfoque de la escalada determinamos que un sistema de segmentos basados en tiempo en la escalada es contraproducente ya que se puede comprometer la técnica y seguridad del ascenso.

2.2.2. Coleccionismo de logros (Pokémon Go)

Pokémon Go(Pok, 2026) Pokémon Go es una aplicación del universo Pokémon centrada en la gamificación del desplazamiento físico con la vinculación de recompensas virtuales y el progreso en la narrativa de un juego. Este sistema permite convertir el esfuerzo en un incentivo para obtener el sentimiento de coleccionismo y descubrimiento. La aplicación es un ejemplo de la implementación del sistema PBL (Points Badges and Leaderboards) previamente definido.

La desventaja principal de esta aplicación es que el objetivo del usuario deja de ser el deporte y pasa a ser la recompensa del juego, creando una posible dependencia negativa en los usuarios.

2.3. Conclusiones del Estado del Arte e identificación de la oportunidad

Para finalizar este capítulo, se presentan las conclusiones obtenidas tras el análisis comparativo entre las aplicaciones, permitiendo identificar la necesidad del mercado y la propuesta de valor de nuestra aplicación.

2.3.1. Síntesis de la problemática actual

Tras analizar las soluciones existentes, se observa que el mercado de aplicaciones de escalada presenta problemas en tres aspectos críticos:

- **Experiencia de Usuario (UX) deficiente:** Aplicaciones como TopLogger o WeClimb ofrecen una gran densidad de datos, pero a costa de interfaces saturadas que dificultan el uso y registro de rutas en la aplicación. La barrera tecnológica aleja al escalador casual que solo busca un registro sencillo de su progresión.
- **Gamificación superficial:** La mayoría de las herramientas se limitan a clasificaciones numéricas (leaderboards) fácilmente manipulables, lo que genera desmotivación. Falta un sistema de recompensas basado en el esfuerzo, la constancia y el "coleccionismo" de hitos personales que no dependa únicamente de ser el mejor del ranking.
- **Dependencia de hardware o geografía:** Existen buenas soluciones como MoonBoard, pero están limitadas a sus muros de escalada específicos, o aplicaciones como WeClimb y TopLogger cuya implantación en España es inexistente. No existe una herramienta universal, intuitiva y "gamificada" que el escalador pueda llevar de un rocódromo a otro.

2.3.2. Identificación de la oportunidad

El análisis de aplicaciones como **Strava** y **Pokémon Go** revela que existe un gran potencial en la adaptación de mecánicas de gamificación de otros sectores a la escalada. La oportunidad reside en desarrollar una solución que combine:

1. **Registro visual y ágil:** Adoptar un sistema híbrido de mapas centiales dinámicos y sistema de tarjetas para minimizar el tiempo de interacción de escalador para reconocer y acceder a la ruta con el móvil durante la sesión.
2. **Gamificación basada en PBL:** Realizar una implementación correcta de el sistema PBL basado en el modelo RAMP de forma que las estadísticas funcionen como los *Points* para mantener la motivación y constancia de los usuarios, además de la implementación de Leaderboards entre grupos de amigos eliminando el factor de los abusos de los sistemas de posicionamiento.

3. **Enfoque social y comunitario:** Fomentar la interacción mediante personalización del perfil y la comparación de estadísticas personales, creando un sentimiento de pertenencia a la comunidad del rocódromo local.

En conclusión, este proyecto se posiciona entre las herramientas de entrenamiento puro y las redes sociales generales, ofreciendo un sistema de entrenamiento que registra datos de las sesiones y motiva activamente al escalador a través de una experiencia lúdica y bien diseñada.

Capítulo 3

Metodología de Desarrollo y Planificación

Uno de los factores más importantes en el desarrollo de un proyecto es el plan y metodología que se va a llevar a cabo para su desarrollo. En este capítulo detallamos cómo se ha definido la metodología de nuestra aplicación llamada *ClimbIt* para lograr un flujo de trabajo funcional con sus herramientas específicas para la gestión y organización que permite ir avanzar eficientemente en las diferentes fases del proyecto.

3.1. Metodología de trabajo: Scrumban

Como metodología de trabajo nos decantamos por utilizar **Scrumban** debido a que consideramos que aplicar Scrum puro podía resultar demasiado estricto especialmente para un equipo de dos personas y en Kanban echamos en falta periodos de trabajo definidos para tener la eficiencia el trabajo con presión de fechas de entrega.

De Scrum nos quedamos con el concepto de *sprints* (iteraciones temporales de desarrollo) de dos semanas aproximadamente y de las técnicas de especificación de requisitos mediante *Historias de usuario* y *Epics* (estructuras para la definición de casos de uso). Por otro lado, de Kanban en lugar de preasignar tareas de forma estricta trabajamos con un sistema *pull* de forma que cada miembro del equipo selecciona la tarea a realizar de la columna *To Do*.

3.2. Proceso de Captura de Requisitos y Definición del Producto

Antes de iniciar la fase de desarrollo e implementación, determinamos que era fundamental comprender en profundidad los problemas a resolver, de manera que no cometiéramos el error de basar la programación en suposiciones no contrastadas. Por lo que dedicamos una fase inicial a la definición estratégica del producto, apoyándonos en herramientas de diseño ágil.

3.2.1. Lienzo de Propuesta de Valor (Value Proposition Canvas)

Con el objetivo de empatizar de manera efectiva con el usuario final, construimos un Value Proposition Canvas, siendo esta una herramienta que nos permite adoptar la perspectiva del escalador y analizar su contexto dividiéndolo en dos bloques principales, el Perfil de cliente y Mapa de valor.

Perfil del cliente : Se enfoca exclusivamente en entender al usuario. Te pones en su situación y describes sus "trabajos"(lo que intenta resolver), sus "dolores." frustraciones (los obstáculos y emociones negativas que encuentra) y sus .^alegrías." ganancias (los resultados y beneficios que desea obtener).

Se identificó que el usuario escalador busca cumplir "trabajos"funcionales (registrar sesiones, ver pistas), sociales (compartir logros) y emocionales (sentirse motivado). Sus principales "frustraciones"(Pains) giran en torno a la dificultad del registro manual, la falta de una plataforma centralizada y la desmotivación por no visualizar su progreso. Por otro lado, sus .^alegrías"(Gains) deseadas son la facilidad para seguir su evolución, recibir feedback y sentirse parte de una comunidad competitiva y sana.

Mapa de Valor Se enfoca en cómo un producto o servicio crea valor en base a los datos del perfil de cliente. Aquí se describen los "productos y servicios.^alivian los "dolores"de tus clientes (Pain Relievers) y cómo generan las .^alegrías"que ellos buscan (Gain creators).

Para responder a este perfil, se determinó que los Pain Relievers se enfocan en ofrecer una interfaz intuitiva, un registro de progreso automatizado y una plataforma segura y centralizada. Finalmente, sus ^creadores de alegrías"(Gain Creators) se centran en proporcionar estadísticas detalladas, un sistema de logros motivador y una comparativa social que impulse al usuario a mejorar e interactuar con sus compañeros.

3.2.2. Visualización del viaje: User Story Map

A partir de la propuesta de valor, se construyó un User Story Map para visualizar el "viaje"completo del usuario dentro de la aplicación, utilizando la plataforma Mural. Un USR (Mapa de Historias de Usuario) es una técnica visual utilizada en la gestión ágil de proyectos para organizar y priorizar las historias de usuario. Este enfoque facilita la comprensión del flujo de trabajo y las interacciones del usuario con el sistema, ayudando a definir y planificar las funcionalidades del producto de manera efectiva.

Este mapa visual resultó muy importante ya que nos ayudó a dividir el proyecto. Trazando líneas horizontales sobre el mapa, pudimos decidir qué funcionalidades eran vitales para el primer prototipo, cuáles entrarían en el Producto Mínimo Viable (MVP) y qué ideas quedaban relegadas a versiones futuras.

3.3. Herramientas de gestión y seguimiento

Para la gestión del proyecto tomamos la decisión de tener las herramientas lo más cercanas posibles para reducir el rozamiento de su uso. Descartamos usar plataformas externas tipo Jira o Trello y centralizamos toda la gestión directamente en **GitHub Projects**, manteniendo los requisitos vinculados al código fuente y las herramientas de despliegue.

El *Product Backlog* se fue rellenando progresivamente con las tareas extraídas del User Story Map. Cada requisito, ya fuera una Historia de Usuario (HU) funcional o una tarea técnica (como configurar la base de datos), se tradujo en una *Issue* dentro del repositorio con información suficiente para no perder contexto.

Atributo	Descripción y uso en el proyecto
Estado	Columna del tablero Kanban (<i>Backlog</i> , <i>To Do</i> , <i>In Progress</i> , <i>Review</i> , <i>Done</i>).
Labels	Categorización visual (<i>bug</i> , <i>backend</i> , <i>frontend</i> , <i>documentación</i> , <i>epic</i> , <i>HU</i> ...).
Estimación	Tamaño (<i>Size</i>) desde XS hasta XL para medir el esfuerzo relativo.
Prioridad	Nivel de urgencia (<i>P0</i> , <i>P1</i> , <i>P2</i>), marcando P0 las tareas críticas de bloqueo.
Milestone	Vinculación directa al <i>sprint</i> actual para seguimiento temporal.
Criterios	Listado de condiciones exactas para considerar la tarea como finalizada.

Tabla 3.1: Anatomía de configuración de una *Issue* en GitHub Projects.

Además, el seguimiento de los cuatro *sprints* se apoyó en un *Roadmap* visual y en gráficos de *Burndown*, que nos permitía visualizar el progreso y detectar si habíamos sobrestimado nuestra capacidad de desarrollo.

3.4. Gestión de la Configuración y Control de Versiones

Implementamos un flujo de trabajo basado en **Git Flow**, estableciendo reglas claras sobre cómo se integra el trabajo de cada desarrollador.

En el repositorio existen dos ramas principales protegidas. La rama **main** se reservó exclusivamente para las versiones estables y liberadas del producto de manera que todo el código de esta rama se subía a producción, mientras que **develop** actuaba como el entorno de preproducción para los nuevos casos de uso. Cada *Issue* del tablero requería la creación de una rama específica (*feature branch*). Al finalizar el desarrollo, esta rama se integraba en **develop** mediante una estrategia de *squash and merge*. Esto nos permitió agrupar decenas de pequeños *commits* en un solo bloque semántico, manteniendo el historial del proyecto limpio y legible.

En paralelo a los ciclos de *sprint*, el despliegue de versiones se gestionó a través de la herramienta de *Releases* de GitHub. Adoptamos el versionado semántico clásico (X.Y.Z) y mantenemos la trazabilidad de los cambios mediante un archivo `CHANGELOG.md` que sigue los estándares de *Keep a Changelog*. Además para mantener la buena praxis el repositorio cuenta desde el inicio con archivos `.gitignore` configurados para el *stack* tecnológico y una guía de instalación en el `README.md`.

3.5. Uso de Inteligencia Artificial como Apoyo al Desarrollo

(por hacer)

Capítulo 4

Análisis y Especificación de Requisitos

Este capítulo detalla a quién va dirigida la aplicación, qué roles intervienen y qué funcionalidades y restricciones debe cubrir la solución.

4.1. Personas

4.1.1. Persona 1: Klaudia, Escaladora Constante

Perfil de referencia para representar a una usuaria frecuente de rocódromo que necesita registrar su progreso y mantener la motivación.

4.2. Roles

4.2.1. Escalador

Rol principal orientado al uso diario de la aplicación para registrar actividad, consultar estadísticas y participar en la gamificación.

4.2.2. Gestor de Rocódromo

Rol encargado de administrar información del centro, rutas y elementos de seguimiento asociados al rocódromo.

4.3. Especificación de funcionalidades (Historias de Usuario / Casos de Uso)

Se agrupan aquí las funcionalidades que la aplicación debe ofrecer, redactadas como historias de usuario o como casos de uso según convenga al nivel de detalle.

4.4. Requisitos no funcionales

Se incluyen las expectativas relativas a rendimiento, seguridad, mantenibilidad, usabilidad y compatibilidad.

4.5. Alcance de las versiones

4.5.1. Requisitos del prototipo (v.0.1.0)

Se describen las capacidades mínimas del primer prototipo funcional.

4.5.2. Requisito del MVP (v.1.0.0)

Se detallan las capacidades que debe alcanzar la primera versión estable y utilizable del producto.

Capítulo 5

Diseño del Sistema y Decisiones Técnicas

En este capítulo se detallan las decisiones técnicas adoptadas para la implementación de Climbit así como la justificación de dichas elecciones frente a las alternativas actuales del mercado. También se detallarán las arquitecturas del frontend y del backend y los patrones de diseño utilizados.

5.1. Decisiones de Diseño y Stack Tecnológico

Para Climbit surgieron varias opciones de diseño para el sistema, pero al final optamos por una separación de responsabilidades para ceder la lógica de negocio y la persistencia de los datos al backend y un frontend SPA (Single Page Application) simple y de fácil navegación que consuma esos datos a través de una API. Desde el comienzo sabíamos que el alcance de esta aplicación era notablemente amplio, por lo que la decisión sobre qué arquitectura utilizar fue previamente meditada. Valoramos tres opciones de diseño para el backend: una arquitectura monolítica, una arquitectura hexagonal y una arquitectura de microservicios.

Para la elección tuvimos en cuenta tres factores principales: la escalabilidad, la mantenibilidad y la complejidad de desarrollo. Una arquitectura monolítica podría haber sido más sencilla de implementar inicialmente, pero a medida que el proyecto creciese, podría habernos llevado a un código difícil de mantener y escalar. Por otro lado, una arquitectura de microservicios nos habría ofrecido grandes ventajas en cuanto a la escalabilidad, pero la curva de aprendizaje y la dificultad que añadía al desarrollo y la necesidad de gestionar múltiples servicios nos hizo descartarla, por el momento, para este proyecto. Al final decidimos que la arquitectura hexagonal nos daba el equilibrio entre dificultad de desarrollo, mantenibilidad y escalabilidad que buscábamos para Climbit, permitiéndonos desacoplar toda la lógica de negocio de factores externos como la base de datos y el frontend, quedando así un sistema más modular y fácil de mantener a largo plazo.

En cuanto al stack tecnológico, para el backend decidimos usar Node.js con Express debido a su popularidad, fiabilidad, facilidad de uso y su amplio ecosistema de librerías. Decidimos trabajar directamente con JavaScript y en formato ESModules y no con TypeScript principalmente por la velocidad de desarrollo y porque Node.js no tiene un soporte nativo para TypeScript. Para la base de datos, descartamos des-

de el principio las bases de datos no relacionales, ya que la naturaleza de los datos de Climbit, con relaciones claras entre escaladores, rocódromos y rutas se adaptaba mejor a un modelo relacional. Ambos habíamos trabajado previamente con bases de datos como MariaDB y MySQL, pero debido a su rendimiento, escalabilidad y su más que creciente popularidad y ecosistema decidimos que para nuestro proyecto, y para las ideas que queremos llegar a implementar en el futuro, PostgreSQL era la mejor opción. Para la comunicación entre el backend y la base de datos, optamos por usar un ORM (Object-Relational Mapping) para facilitar el desarrollo, así como una capa extra de seguridad. Después de evaluar varias opciones decidimos que Sequelize, debido a su diseño nativo para JavaScript, era la opción más indicada. Otros ORMs como TypeORM o Prisma también eran opciones válidas, pero al estar más orientados a TypeScript no podríamos aprovechar al máximo sus características, así que decidimos que Sequelize era la mejor opción para nuestro proyecto.

–Falta hablar del uso de tokens y de ffmpeg para la conversión a webp

Para el frontend sabíamos que queríamos una aplicación web, pero que fuera PWA (Progressive Web App), para que los usuarios de cualquier dispositivo pudiera acceder a ella y descargarla sin la necesidad de tener que crear una aplicación nativa para cada plataforma. Desde el principio fuimos reticentes al uso de frameworks frontend. Después de valorar las ventajas que nos ofrecían frente la curva de aprendizaje y la complejidad añadida que suponían, decidimos optar por una solución más ligera y sencilla: utilizar HTML, CSS y JavaScript puro, ayudados de Bootstrap para mantener un diseño consistente. Esto nos permitió tener un control total sobre el código y una velocidad inicial de desarrollo mucho mayor, ya que los dos habíamos desarrollado previamente con estas tecnologías. Por otro lado, si nos supuso un reto la implementación de ciertas funcionalidades, en concreto el enrutamiento de las vistas o la implementación manual de componentes personalizados, que es algo que los frameworks manejan de forma nativa y que nosotros tuvimos que implementar desde cero, pero al final creemos que fue la decisión correcta para el proyecto.

5.2. Diseño de la Arquitectura

5.2.1. Backend

Una vez elegida la arquitectura hexagonal para el backend, se diseñó la estructura del proyecto siguiendo los principios de esta arquitectura y aplicando una serie de patrones de diseño que nos permitieran mantener un código limpio, sencillo y modular, completamente independiente entre las diferentes capas. La arquitectura hexagonal organiza el código en capas definidas, donde cada una tiene una responsabilidad clara. La capa de dominio se encarga de la lógica de negocio y las reglas del sistema, donde se definen todos los contratos y validaciones sobre los objetos que se procesan. La capa de infraestructura se encarga de la comunicación con la base de datos, que nosotros gestionamos a través de Sequelize, así como los modelos que representan las entidades del sistema, las migraciones y las seeds. La capa de aplicación se encarga de definir la lógica de los use cases, orquestando la interacción entre la capa de dominio y la capa de infraestructura. Por último, la capa de pre-

sentación, que nosotros hemos llamado interfaz, se encarga de la comunicación con el frontend a través de una API REST, definiendo los endpoints, los controladores que gestionan las peticiones y respuestas y los middlewares que nos ayudan a la autenticación de tokens, formatos de datos y validaciones tanto de datos como de archivos.

Diagrama de la arquitectura hexagonal del backend con ejemplo de Climbit

A lo largo del desarrollo nos encontramos con diversas situaciones que nos llevaron a aplicar diferentes patrones de diseño que nos ayudaron a obtener un código más limpio, modular y fácil de mantener. Los patrones de diseño que hemos utilizado son los siguientes:

- Patrón Repository: Este patrón propuesto por Martin Fowler consiste en crear una interfaz entre la lógica de negocio y la capa de persistencia, en nuestro caso, nuestra base de datos PostgreSQL a través de Sequelize, de manera que nos permite desacoplar el negocio de la infraestructura, manteniendo así un código más mantenible en el tiempo. En nuestro caso, como se puede ver en la Figura XX, la capa de dominio define la interfaz del repositorio en la que se establecen los métodos que se necesitan para realizar las diferentes operaciones y la capa de infraestructura implementa dicha interfaz con la que hablar con la base de datos. De esta manera, la capa de aplicación solo se encarga de definir la lógica de los distintos casos de uso.

Figura del diagrama de patrón repository

- Patrón Data Mapper: También definido por Martin Fowler, el patrón Data Mapper se encarga de mapear los objetos del dominio a los objetos de la base de datos y viceversa. Es útil cuando el dominio y la base de datos tienen estructuras similares pero no idénticas. En nuestro caso, aunque la estructura de los objetos del dominio y los modelos son prácticamente idénticos, al transformar datos no tipados por estar definidos en JavaScript y al trabajar con PostgreSQL, que añade ciertos campos automáticamente, como los campos de CreatedAt y UpdatedAt, decidimos implementar este patrón pero solo para mapear los objetos recibidos por el modelo al dominio. De esta manera nos aseguramos que no se producían errores al trabajar con objetos con campos inesperados.

- Patrón Adapter: El patrón Adapter es uno de los patrones clásicos introducidos por Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, también conocidos como los "Gang of Four". Se encarga de adaptar distintas interfaces para que puedan trabajar entre ellas. En nuestro caso, lo hemos utilizado para adaptar distintos errores que pueden surgir en las diferentes capas, como se puede observar en la Figura XX, para centralizar el manejo de estos errores en un único lugar, permitiéndonos así mantener una consistencia a la hora de tratarlos distintos errores que pueden surgir.

Figura del diagrama de patrón adapter

- Patrón Singleton: Introducido también por "The Gang of Four" en su libro "Design Patterns: Elements of Reusable Object-Oriented Software", el patrón Singleton sirve para garantizar que una clase tenga únicamente una instancia al mismo tiempo a lo largo de la ejecución de la aplicación. En nuestro caso, lo hemos utilizado, por ejemplo, para mantener una única conexión a la base de datos, evitando así posibles problemas y errores relacionados con múltiples conexiones abiertas, pero también lo hemos utilizado en otros puntos que detallaremos más adelante en la implementación

de la aplicación.

La inversión de control se basa en el principio de que los objetos no deben crear sus propias dependencias, sino que deben recibirlas desde el exterior. Esto permite un mayor desacoplamiento entre los componentes del sistema, facilitando la mantenibilidad y la escalabilidad. La inyección de dependencias es una técnica para implementar la inversión de control, donde las dependencias se proporcionan a un objeto en lugar de ser creadas por el propio objeto. Esto se puede lograr a través de diferentes métodos, como la inyección por constructor, la inyección por setter o la inyección por interfaz. En nuestro proyecto, hemos utilizado principalmente la inyección por constructor para proporcionar las dependencias necesarias a nuestros objetos, lo que nos ha permitido mantener un código más limpio y modular.

Inyección de dependencias e Inversión de control: La inversión de control es un principio abstracto popularizado por Robert C. Martin, también conocido como "Uncle Bob", pero ideada inicialmente por Martin Fowler, que viene a referir que los objetos no deben crear sus propias dependencias, sino que le tienen que venir dadas de fuera. Por otro lado, la inyección de dependencias una de las técnicas para implementar este principio. En nuestra aplicación hemos utilizado la inyección de dependencias a través de la inyección por constructor transversalmente en toda la aplicación. El controlador recibe por inyección los casos de uso que necesita para gestionar las peticiones, los casos de uso reciben por inyección los repositorios que necesitan para realizar sus operaciones y los repositorios reciben por inyección los modelos que necesitan para hablar con la base de datos. De esta manera, hemos conseguido un código completamente desacoplado muy en la línea de la filosofía de la arquitectura hexagonal.

- Patrón Comando (falta)
- Patrón Fachada (falta)

5.2.2. Frontend

Se describe cómo se organiza la interfaz de usuario y su comunicación con el backend.

5.3. Diseño de la Base de Datos

Se presenta el modelo de datos y la lógica de persistencia asociada.

5.4. Diseño de la UI y UX

Se recogen los criterios de interacción visual, flujo de pantallas y experiencia de uso.

Capítulo 6

Implementación y Desarrollo

Este capítulo documenta la construcción práctica de la solución y las decisiones tomadas durante su desarrollo.

6.1. Desarrollo del Backend

Se describen los módulos principales del servidor, sus responsabilidades y la lógica de negocio implementada.

6.2. Desarrollo del Frontend

Se explica la implementación de la interfaz y la integración con los servicios de la aplicación.

6.3. Desafíos técnicos encontrados y soluciones

Se recogen los problemas técnicos relevantes que aparecieron durante el desarrollo y cómo se resolvieron.

Capítulo 7

Integración Continua, Despliegue y DevOps (CI/CD)

En este capítulo se resume cómo se automatiza la entrega del software y cómo se preparan los distintos entornos del proyecto.

7.1. Diseño del Pipeline de CI/CD

Se describen las etapas del pipeline, sus validaciones y su finalidad dentro del ciclo de entrega.

7.2. Entornos de desarrollo, preproducción y producción

Se diferencian los entornos utilizados y el propósito de cada uno.

7.3. Despliegue de la infraestructura

Se explica la preparación y publicación de la infraestructura necesaria para ejecutar la solución.

7.4. Empaquetado y Subida a Google Play

Se documenta el proceso de empaquetado de la aplicación móvil y su preparación para distribución.

Capítulo 8

Pruebas y Validación

Este capítulo reúne la estrategia de verificación de la solución y la validación de la utilidad de la aplicación.

8.1. Estrategia de pruebas técnicas

Se definen los tipos de pruebas aplicadas para comprobar el correcto funcionamiento del sistema.

8.2. Pruebas con usuarios reales (User Testing)

Se describe la validación realizada con usuarios para comprobar usabilidad y adecuación al contexto real.

8.3. Análisis de Resultados y Mejoras aplicadas

Se sintetizan los resultados obtenidos y los cambios introducidos a partir de ellos.

Capítulo 9

Conclusiones y Trabajo Futuro

Conclusiones del trabajo y líneas de trabajo futuro.

Antes de la entrega de actas de cada convocatoria, en el plazo que se indica en el calendario de los trabajos de fin de grado, el estudiante entregará en el Campus Virtual la versión final de la memoria en PDF.

Introduction

Introduction to the subject area. This chapter contains the translation of Chapter 1.

Conclusions and Future Work

Conclusions and future lines of work. This chapter contains the translation of Chapter 9.

Contribuciones Personales

En caso de trabajos no unipersonales, cada participante indicará en la memoria su contribución al proyecto con una extensión de al menos dos páginas por cada uno de los participantes.

En caso de trabajo unipersonal, elimina esta página en el fichero `TFGTeXiS.tex` (comenta o borra la línea `\include{Capitulos/ContribucionesPersonales}`).

Estudiante 1

Al menos dos páginas con las contribuciones del estudiante 1.

Estudiante 2

Al menos dos páginas con las contribuciones del estudiante 2. En caso de que haya más estudiantes, copia y pega una de estas secciones.

Bibliografía

*Y así, del mucho leer y del poco dormir, se
le secó el cerebro de manera que vino a
perder el juicio.*

(modificar en Cascaras\bibliografia.tex)

Miguel de Cervantes Saavedra

Kaya climbing app. 2026.

Moonboard. 2026.

Pokémon go. 2026.

Strava. 2026.

Toplogger. 2026.

Weclimb. 2026.

Apéndice **A**

Título del Apéndice A

Los apéndices son secciones al final del documento en las que se agrega texto con el objetivo de ampliar los contenidos del documento principal.

Apéndice	B
----------	----------

Título del Apéndice B

Se pueden añadir los apéndices que se consideren oportunos.

Este texto se puede encontrar en el fichero Cascaras/fin.tex. Si deseas eliminarlo, basta con comentar la línea correspondiente al final del fichero TFGTeXiS.tex.

*–¿Qué te parece desto, Sancho? – Dijo Don Quijote –
Bien podrán los encantadores quitarme la ventura,
pero el esfuerzo y el ánimo, será imposible.*

*Segunda parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

*–Buena está – dijo Sancho –; fírmela vuestra merced.
–No es menester firmarla – dijo Don Quijote–,
sino solamente poner mi rúbrica.*

*Primera parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

